Saved references — AERIS-10 radar, Minev Quantum Notebook, customcst FDTD

Study notes and indexes, RFMW-ANT repository (projects 09–11)

23 October 2026

- Part 1: AERIS-10 open-source X-band phased-array radar
- Part 2: Zlatko Minev's Quantum Notebook (saved copy)
- Part 3: customcst, a CST-style FDTD solver (saved copy)

Part 1: AERIS-10 open-source X-band phased-array radar

Source: https://github.com/NawfalMotii79/PLFM_RADAR, commit `62d4c55` (14 Oct 2026), by Nawfal Motii and contributors. Hardware: CERN-OHL-P v2. Software/firmware: MIT. Status upstream: **alpha**. The FPGA builds are timing-clean and the regressions pass, but the radar hardware had not been brought up yet (the docs describe the board-day plan).

Numbers marked *(derived)* come from `analysis/aeris10_numbers.py`, which recomputes them from the firmware constants. Everything else is quoted from the upstream files named in brackets.

---

1. The system in one paragraph

AERIS-10 is an open-source **10.5 GHz (X-band) pulse-LFM phased-array radar**. A DAC makes a 20 MHz chirp, which is mixed up to 10.5 GHz and transmitted through 16 front-end channels. Each channel has an ADTR1107 T/R chip, and the 16 channels are steered electronically by four 4-channel ADAR1000 beamformers (±45°). A stepper motor and slip ring add 360° mechanical rotation. The echo is mixed down to a 120 MHz IF and sampled at 400 MSPS by an 8-bit AD9484. A Xilinx Artix-7 FPGA then does DDC → pulse compression → range decimation → MTI → Doppler FFT → CFAR and streams the detections over USB to a Python GUI. An STM32F746 sequences the power rails, programs the clocks, LOs and beamformers, closes the PA-bias loop, and reads GPS, IMU and temperature.

Version	Antenna	Power	Range
AERIS-10N "Nexus"	8 × 16 patch array on RO4350B	≈ 1 W × 16	3 km
AERIS-10E "Extended"	32 × 16 alumina-filled slotted-waveguide array	10 W × 16 (QPA2962 GaN)	20 km

2. Frequency plan and waveform

- **Clocks.** AD9523-1 jitter cleaner from a 100 MHz VCXO feeds both ADF4382A LOs, the DAC, the ADC and the FPGA. This keeps everything phase-coherent, which Doppler processing needs. [README, main.cpp `configure_ad9523`]
- **LOs.** TX LO 10.5 GHz, RX LO 10.38 GHz, so the IF is 120 MHz. [adf4382a_manager.h:35-36]
- **Chirp.** The DAC runs at 120 MSPS and sweeps 30 → 10 MHz (B = 20 MHz). The two LTC5552 mixers handle the up- and down-conversion. [plfm_chirp_controller.v:36-45]

- **Two pulses per slot.** Each “chirp slot” holds a long pulse and a short pulse:

Pulse	Length	Listen time	Purpose
Long	30 µs	137 µs	far targets, with big pulse-compression gain
Guard	175.4 µs	—	separates the two pulses
Short	0.5 µs	174.5 µs	near targets inside the long pulse’s blind zone

- **Performance** *(derived)*:

Quantity	Value
Range resolution c/2B	**7.5 m**
Long-pulse time–bandwidth product	600, i.e. **27.8 dB** compression gain
Long-pulse blind zone (TX on)	**4.5 km**
Short-pulse blind zone	75 m
Long listen window	20.5 km (matches the 20 km version)
Slot period	517.4 µs, so PRF = **1932.7 Hz**
One frame (32 slots)	16.6 ms
Unambiguous velocity	±13.8 m/s at λ = 28.6 mm
Velocity resolution (16-point FFT)	1.7 m/s

- **Scan.** 32 chirps per beam position × 31 elevations × 50 azimuths. A full electronic raster is 1550 beams, about **25.7 s per scan** *(derived)*. [radar_mode_controller.v:34-36]

3. FPGA signal chain (9_Firmware/9_2_FPGA)

Stage	Module	Key facts
ADC capture	`ad9484_interface_400m.v`, `adc_clk_mmcm.v`, `cdc_modules.v`	400 MHz domain, MMCM jitter cleaning (Build 20), toggle/handshake CDC into the 100 MHz processing domain
DDC	`ddc_400m.v`, `nco_400m_enhanced.v`	8-bit ADC, 16-bit NCO, 18-bit mixer, phase increment 0x4CCCCCCD = 120 MHz / 400 MHz, dithering
Decimation	`cic_decimator_4x_enhanced.v`, `fir_lowpass.v`	5-stage CIC ÷4 (48-bit accumulators, CREG-pipelined DSP48 combs), then a 32-tap FIR (18-bit) at 100 MSPS
Pulse compression	`matched_filter_processing_chain.v`, `matched_filter_multi_segment.v`, `*.mem`	frequency domain: 1024-point FFT of the signal and reference, conjugate multiply, IFFT; the 3000-sample long chirp is processed in 4 overlapping segments (128-sample overlap); the reference chirps are in `long_chirp_seg*_i/q.mem`, `short_chirp_*.mem`
FFT core	`fft_engine.v`, `xfft_16.v`	home-made radix-2; 4-cycle butterfly (+20 % throughput), twiddle index by barrel shift (−1 DSP)
Range decimation	`range_bin_decimator.v`	1024 → 64 bins (÷16), start-bin skip, watchdog; ≈ 24 m bins and a 1.5 km window per mode *(derived)*

Stage	Module	Key facts
Clutter	`mti_canceller.v` + DC notch in top	2-pulse canceller H(z) = 1 − z^{-1}; post-Doppler notch of ±N bins around zero (registers 0x26/0x27)
Doppler	`doppler_processor.v`	**staggered PRF**: two 16-point Hamming FFTs, one per sub-frame (16 long-PRI + 16 short-PRI), because one 32-point FFT over non-uniform sampling would be invalid; outputs tagged {sub_frame, bin} for later ambiguity resolution
Detection	`cfar_ca.v`	CA / GO / SO-CFAR on |I|+|Q|, 64 × 32 BRAM map, up to 8 guard and 16 training cells, alpha in Q4.4 pre-divided by the cell count (example: Pfa 1e-4 with T = 8 per side → alpha ≈ 0x05); 3-cycle pipelined CUT (THR→MUL→CMP), about 85 µs per frame
Gain	`rx_gain_control.v` + STM32 `ADAR1000_AGC.cpp`	hybrid AGC: FPGA saturation flag → MCU outer loop → ADAR1000 RX VGA gain; target/attack/decay/hold-off registers 0x28–0x2C
Host link	`usb_data_interface.v` (FT601, 32-bit USB 3), `usb_data_interface_ft2232h.v` (8-bit USB 2)	11-byte data packet `AA, range I/Q, Doppler re/im, det, 55`; 26-byte status packet `BB … 55`; 4-byte commands {opcode, addr, value_hi, value_lo}
Self test	`fpga_self_test.v`	opcode 0x30; status readback with 0x31 or 0xFF

Host register map (radar_system_top.v ≈ 951-997):

Opcode	Function
0x01	mode
0x02	trigger
0x03	threshold
0x04	stream control
0x10–0x15	chirp timing and chirps per elevation
0x16	digital gain shift
0x20	range mode
0x21–0x25	CFAR guard / train / alpha / mode / enable
0x26–0x27	MTI and DC notch
0x28–0x2C	AGC
0x30, 0x31, 0xFF	self test and status

Targets. Production is an xc7a200t-2fbg484i. There is also a 50T board (FT2232H), plus Trenz TE0712/TE0713 + TE0701 dev wrappers with separate XDC files. Build 25 (v0.1.6-mti) uses 9252 LUTs, 12488 FFs, 17 BRAM and 142 DSP48 at 0.75 W, with WNS +0.132 ns. [docs/release-notes.html]

4. STM32F746 firmware (9_Firmware/9_1_Microcontroller)

- Bare-metal HAL with Analog Devices **no-OS** drivers for the AD9523, ADF4382A and ADAR1000. `main.cpp` is about 2900 lines.
- **Power sequencing.** Follows `3_Power Management/Power Management V6.xlsx`. The IWDG watchdog, `Emergency_Stop` (cuts the PA rails), a temperature-max guard and periodic Idq re-reads were added under "Gap 3".
- **PA bias loop.** 16 channels, each with a 5 mΩ shunt and an INA241 (×50) read by two ADS7830 I²C ADCs, and Vg set by two DAC5578. Idq is calibrated closed-loop at boot.
- **Thermal.** 8 thermistors on a third ADS7830, with one fan GPIO.
- **Beam steering.** `ADAR1000Manager::calculatePhaseSettings` uses λ/2 spacing at 10.5 GHz and 7-bit words (128 states, 2.8125°). There are also beam-sequence and sweep APIs and custom 16-element patterns.
- **Sensors.** UM982 GPS (TinyGPS++), GY-85 IMU (pitch/roll correction of target coordinates), BMP180 barometer, stepper motor.
- **Tests.** `tests/` holds host-side unit tests with HAL/driver mocks. **One test per fixed bug** (bug1… bug15, the gap3 safety tests, GPS, AGC), run with a Makefile. This is an excellent pattern.

5. GUI (9_Firmware/9_3_GUI)

- `radar_protocol.py` is the pure-logic packet parser and command builder, with no GUI imports, so it is testable.
- `GUI_V65_Tk.py` is a bring-up dashboard:
  - live range–Doppler heat map with the CFAR overlay;
  - waterfall;
  - host commands;
  - HDF5 recording, replay and a demo mode.
- `v7/` is a PyQt6 application in modules: `dashboard`, `hardware`, `map_widget`, `processing`, `software_fpga.py` (a software model of the FPGA chain), `replay`, `agc_sim`.
- `smoke_test.py` triggers the FPGA self test.

6. Antennas and RF simulations (5_Simulations, docs/*.pdf)

- **Patch element** (openEMS FDTD, docs/AERIS_Antenna_Report.pdf):

Quantity	Value
Substrate	RO4350B, εr 3.48, 0.102 mm
Resonance	10.495 GHz
S11	−30.6 dB
Bandwidth	50 MHz (0.48 %)
Directivity	7.19 dBi
Efficiency	56.7 %
128-element array gain	≈ 25.8 dBi

  The 50 MHz bandwidth is narrow, but enough for a 20 MHz chirp.

- **Slotted waveguide array** (`Slotted_DielectricFilled_Waveguide.m`, `wg_alumina_slotted_openems.m`, `array_pattern_Kaiser25dB_like.py`, `slot_layout_taper32.csv`):
  - alumina-filled guide (εr 9.8), a = 8.5 mm, b = 3.5 mm;
  - 32 longitudinal offset slots per column, 16 columns at 14.28 mm;
  - Kaiser β = 1.65 amplitude taper set through the slot offsets (0.22–0.30 a);
  - **standing-wave design** (slot pitch λg/2, backshort λg/4 behind the feed), so the beam is fixed at broadside in the waveguide plane and steered electronically across the columns.

- **Other ADS / openEMS work:** IF band-pass filters (including a balanced one), stub BPF, DAC reconstruction filter, anti-alias filter, RF switch, via fencing, QPA2962 PA match.

7. Findings worth reporting upstream (checked by us)

1. **Guide wavelength in the alumina-filled waveguide ignores εr.**
   - Where: `Slotted_DielectricFilled_Waveguide.m` line 22 and `wg_alumina_slotted_openems.m` line 64 both compute λg = λ0 / √(1 − (fc/f)²) with the free-space λ0.
   - The correct formula for a filled guide uses λ0/√εr: λg = (λ0/√εr) / √(1 − (fc/f)²) = **10.81 mm, not 33.83 mm** *(derived)*.
   - Consequence: the slot pitch λg/2 should be 5.40 mm, not 16.9 mm. With 16.9 mm the slots are about 1.6 λg apart, so they are not in phase and the array will not radiate at broadside as intended.
   - The slot-length rule 0.47 λg is tied to the wrong λg too. A slot backed by the dielectric resonates somewhere between λ0/2 ≈ 14.3 mm (air) and about λ0 / (2√((εr+1)/2)) ≈ 6 mm. So re-derive the length and the offset-conductance table with an openEMS / FEM unit-cell sweep instead of scaling it.
   - `slot_layout_taper32.csv` (z pitch 16.915 mm) inherits the error.
2. **Beam-steering phases repeat every 4 elements.**
   - Where: `ADAR1000Manager::setBeamAngle` computes phases for i = 0…3 only and writes the same 4 words to every ADAR1000.
   - If the 16 channels form one line across the four chips, elements 4–15 get the wrong progression (0, φ, 2φ, 3φ, 0, φ, …). The correct 16-element words are in `analysis/aeris10_beam_phases.csv`.
   - This is only correct if each ADAR1000 drives a separate row or column, which the schematic should confirm.
3. **The long pulse is blind out to 4.5 km**, and its 137 µs listen window ends at 20.5 km. Targets between 75 m and 4.5 km depend on the 0.5 µs short pulse, which has no compression gain, so sensitivity is much lower there. Worth stating clearly in the range spec of the 3 km version.

8. Techniques worth reusing in our projects

- **Golden-model co-simulation.** Python reference vs RTL, bit-exact on recorded ADC data (5137 checks), plus a 46-check end-to-end testbench and a regression script.
- **Bug-numbered regression tests** for MCU firmware, with HAL mocks on the host.
- **Clean separation.** DSP core modules stay board-agnostic. Per-board top wrappers and XDCs live separately, and bring-up starts with a minimal heartbeat image.
- **Timing closure, documented per build.** WNS/WHS and resources are logged in the release notes, the cause of a timing failure is pinned down (Build 23 CFAR combinational path → 3-stage pipeline), and DSP48 CREG/PREG pipelining is used.
- **Radar-architecture choices:**
  - long + short pulse to cover the blind zone;
  - staggered PRF with separate FFTs;
  - MTI + DC notch before CFAR;
  - hybrid FPGA/MCU AGC.
- **Bring-up discipline.** Pre-arrival gates, a smoke-test order, explicit abort criteria (docs/bring-up.html, board-day-worksheet.html).

9. Links to this repository

- `projects/08_60ghz-serpentine-lwa_TAP2026` is also a slotted waveguide array, but a *travelling-wave* one whose beam steers with frequency. AERIS-10 uses the *standing-wave* (λg/2, broadside) variant and steers with phase shifters instead.

- `docs/GDS2OPENEMS.md` / `docs/HERTZON.md`: the same openEMS / FEM tools can re-simulate the AERIS-10 patch and the corrected alumina slot array.

Part 2: Zlatko Minev's Quantum Notebook (saved copy)

Verbatim copy of **zlatko-minev/zlatko-minev-quantum-repository** (website: https://zlatko-minev.github.io/zlatko-minev-quantum-repository/), commit `afdaf49` (2026-10-10). It holds 76 talks, lectures and notes by Zlatko K. Minev (2011–2026) on superconducting qubits, cQED, quantum hardware design (e.g. energy-participation-ratio / pyEPR, Qiskit Metal), error mitigation and quantum physics.

Licence

CC BY-NC-ND 4.0, © 2011–2026 Zlatko K. Minev (see `upstream_snapshot/LICENSE.md`).

- **Allowed:** share verbatim copies, non-commercially, with attribution.
- **Not allowed:** editing the decks, or using them commercially.

This copy is unmodified except for two renames, so that AI coding assistants working in this repository do not load the upstream's own agent instructions:

- `CLAUDE.md` → `UPSTREAM_CLAUDE.md`
- `.claude/` → `_upstream_dot_claude/`

Cite the decks with `upstream_snapshot/CITATION.cff` or the BibTeX on the website.

Browse

- Open the PDFs under `upstream_snapshot/content - */`.
- Or run the site locally: `cd upstream_snapshot && python3 tools/build_site.py --serve` (port 8000).
- `catalog.json` lists every item with year, venue, tags and description.

Index

lectures (41)

Year	Title	Venue	Pages	File
2026	Superconducting Qubits 101: cQED Quantum Hardware	Quantum Device Workshop (Beginner)	81	pdf
2025	A Prelude to Practical Quantum Computing	Global Hackathon AI For Good	82	pdf
2025	Life & Lessons as a Scientist in "Real Life" (career talk)	Qubit x Qubit Capstone	17	pdf
2025	Your First Superconducting Qubit: Intro to cQED, Part I	Flagship Quantum Device Workshop (Beginner)	57	pdf
2025	Your First Superconducting Qubit: Intro to cQED, Part II	Flagship Quantum Device Workshop (Beginner)	58	pdf
2024	Introduction to Noise and Quantum Error Mitigation, Lecture 1	Near-Term Quantum Algorithms Summer School	66	pdf

Year	Title	Venue	Pages	File
2024	Introduction to Noise and Quantum Error Mitigation, Lecture 2	Near-Term Quantum Algorithms Summer School	107	pdf
2023	Cheat Sheet: Digital Quantum Circuits, T11.7 (bonus)	Boulder Summer School	2	pdf
2023	Cheat Sheet: Pictorial Digital Quantum Circuits, T11.7 (bonus)	Qiskit Global Summer School	2	pdf
2023	Cheat Sheet: Tail Bounds, T11.9 (bonus)	Boulder Summer School	2	pdf
2023	Coherent ZZ Error (bonus)	Qiskit Global Summer School	1	pdf
2023	Estimating Observables from Bitstrings, T21 (bonus)	Qiskit Global Summer School	2	pdf
2023	Hello Quantum World! (career talk)	Guest lecture, Sophia Economou's class	88	pdf
2023	Intro to Quantum Measurements, Part I: Ideal Measurements	None	52	pdf
2023	Intro to Quantum Noise (main lecture)	Qiskit Global Summer School	128	pdf
2023	PEC Derivation: Full Derivation (C65C)	Boulder Summer School	4	pdf
2023	PEC Derivation: Single Qubit, Simple (C65B)	Boulder Summer School	4	pdf
2023	Quantum Error Mitigation, Lecture 1	Boulder Summer School	71	pdf
2023	Quantum Error Mitigation, Lecture 2	Boulder Summer School	82	pdf
2023	Quantum Error Mitigation, Lecture 3	Boulder Summer School	82	pdf
2022	Coherent ZZ Error (bonus)	Qiskit Global Summer School	1	pdf
2022	Incoherent Noise (note)	Qiskit Global Summer School	1	pdf
2022	Intro to Quantum Noise (main lecture)	Qiskit Global Summer School	99	pdf
2022	Intro to Real Quantum Computers and Noise	Vanderbilt Summer Lecture	93	pdf
2022	Measurement (note)	Qiskit Global Summer School	3	pdf
2022	Measurement Noise (note)	Qiskit Global Summer School	2	pdf
2022	Primer on Pauli Twirling, Part I	None	24	pdf
2022	Primer on Pauli Twirling, Part II	None	19	pdf

Year	Title	Venue	Pages	File
2022	Primer on Pauli Twirling, Part III	None	16	pdf
2022	Primer on Pauli Twirling, Part IV	None	16	pdf
2022	State Preparation Noise (bonus)	Qiskit Global Summer School	1	pdf
2021	Coherent ZZ Error	Qiskit Global Summer School	1	pdf
2021	Incoherent Noise	Qiskit Global Summer School	2	pdf
2021	Intro to Quantum Noise	Qiskit Global Summer School	65	pdf
2021	Measurement Noise	Qiskit Global Summer School	1	pdf
2021	Projection Noise	Qiskit Global Summer School	2	pdf
2021	State Preparation Noise	Qiskit Global Summer School	1	pdf
2021	Superconducting Qubits 101: cQED Quantum Hardware	CEC ICMC	113	pdf
2021	Superconducting Qubits 101: cQED Quantum Hardware	QIQT Summer School	84	pdf
2020	Superconducting Qubits 101: Quantum Hardware & cQED, Day 1	Qiskit Global Summer School	80	pdf
2020	Superconducting Qubits 101: Quantum Hardware & cQED, Day 2	Qiskit Global Summer School	73	pdf

notes (8)

Year	Title	Venue	Pages	File
2023	Introduction to Many-Body Physics: Exploring Spin Models for Quantum Computing Scientists	None	10	pdf
2023	Simple Noise Models and Gate Fidelity	None	10	pdf
2022	Bounding a Sum of Operators (à la Watrous)	None	2	pdf
2022	Estimating Observables from Sampled Bit Strings	None	2	pdf
None	Are Mixed States and Random Sampling of	None	11	pdf

Year	Title	Venue	Pages	File
	States Equivalent?			
None	Concentration Inequalities and Tail Bounds	None	2	pdf
None	Pictorial Digital Quantum Circuits	None	5	pdf
None	Probabilistic Error Cancellation: Paper Notes	None	2	pdf

research (27)

Year	Title	Venue	Pages	File
2026	OTOC-NMR Experiment: Error Mitigation, Modeling and Results	APS March Meeting	14	pdf
2025	Machine Learning for Practical Quantum Error Mitigation (ML-QEM)	Aspen AI plus Quantum	60	pdf
2025	Overview of Quantum Hardware Design: Energy, Circuits, and Metal	Flagship Quantum Device Workshop (Advanced)	93	pdf
2024	Fibonacci Anyons and String-Net Condensation; Entanglement-Enhanced Learning of Quantum Processes at Scale	CIFAR	47	pdf
2024	Uncovering Local Integrability in Quantum Many-Body Dynamics	APS March Meeting (invited)	63	pdf
2023	Machine Learning for Practical Quantum Error Mitigation (ML-QEM)	Qiskit Seminar	19	pdf
2022	Overview of Quantum Hardware Design: Energy, Circuits, and Metal (Qiskit Metal)	Chicago Quantum Exchange, NanoKorea	55	pdf
2022	Overview of Quantum Hardware Design: Qiskit Metal, from Fab-It to Qubit	NQI	88	pdf
2022	Qiskit Metal: Quantum Device Design and Analysis (Q-EDA)	QED-C	30	pdf
2022	To Learn and Cancel Quantum Noise: PEC	IBM Quantum Qiskit Seminar	64	pdf

Year	Title	Venue	Pages	File
	with Sparse Pauli-Lindblad Models			
2022	To Learn and Cancel Quantum Noise: PEC with Sparse Pauli-Lindblad Models	KITP	71	pdf
2021	Qiskit Metal: Overview of Quantum Hardware Design (Energy, Circuits, Metal)	IEEE Quantum Week	31	pdf
2021	Quantum Hardware Design: Energy, Circuits, Metal (Qiskit Metal, EPR)	Qiskit Seminar	123	pdf
2021	To Catch and Reverse a Quantum Jump Mid-Flight	None	113	pdf
2021	To Design Quantum Hardware - Qiskit Metal and Energy Participation Ratio (EPR) for Superconducting Qubit Design	None	110	pdf
2020	To Catch and Reverse a Quantum Jump Mid-Flight (broad talk)	MIT Technology Review EmTech (TR35)	11	pdf
2020	pyEPR Working Group Meeting #1	pyEPR Meeting	19	pdf
2019	Advancing Quantum Sensing via Quantum Control (keynote)	Padua, Italy	95	pdf
2019	Energy Participation Ratio (EPR) for Superconducting Qubit Design	None	79	pdf
2015	2.5D cQED Coherence: Exploring Drawbridge Transmon Coherence	None	26	pdf
2014	2.5D Circuit Quantum Electrodynamics (cQED): Whispering Gallery Modes	None	32	pdf
2013	Superconducting Whispering Gallery Mode Resonators	APS March Meeting	11	pdf
2012	Superconducting Whispering Gallery Modes	None	30	pdf
2012	Wafer Assembled Whispering Gallery Mode Resonator	None	22	pdf

Year	Title	Venue	Pages	File
2012	Whispering Gallery Modes	None	25	pdf
2011	Josephson Junction Arrays: Parasitic Capacitances	None	37	pdf
2011	Optimizing Josephson Parametric Amplifiers for Fast, High-Fidelity Qubit Readout	APS March Meeting (poster)	1	pdf

Part 3: customcst, a CST-style FDTD solver (saved copy)

Copy of the `customcst/` folder of **inhardwetrust/Electrodinamics_exp**, commit `1d269c6` (2026-10-22), with the repository's `LICENSE`.

What it is

An attempt to rebuild part of CST Studio's time-domain workflow from scratch, in about 13k lines of Python: 1. import a STEP model; 2. assign materials and ports per body; 3. build a non-uniform rectilinear (hexahedral) grid; 4. run a 3D FDTD; 5. read S11 like on a VNA.

The upstream README states that all of the code was written with Claude, with the author directing.

The code is built in levels, bottom-up (upstream `ARCHITECTURE.md`):

Level	Folder	Role
0	`geometry/`	STEP import via OpenCASCADE (`cadquery-ocp`) into triangulated bodies; KiCad STEP handling (copper merged per layer, vias as one body), holes, ray casting
1	`project/`	project model: materials per body, ports, frequency range, boundaries (`model.toml`)
2	`mesh/rectilinear.py`	compute grid: fixpoints at planar CAD faces, max step λ_min/N per material, grading limit
3	`discretize/`	ε, σ, μ and PEC masks on the Yee grid (staircase; conformal is planned)
4	`solver/`	non-uniform Yee FDTD with numba kernels, CPML, lumped ports and loads, 2D waveguide-port mode solver (`port_mode.py`), S-parameters (`sparams.py`), VNA-style display (`vna.py`)
5	monitors	S-matrix and Z_in done; DFT field monitors and far field still planned
6	`view/`	PySide6 + vispy GUI: geometry, sections with the mesh, results

The models worked through upstream: - a microstrip with discrete ports; - a 50 Ω coax with waveguide ports (Z0 = 49.5 Ω from the port solver); - an SMA connector on a 4-layer PCB with a grounded coplanar line and a 50 Ω lumped load, at about −30 dB S11 over 4–6 GHz; - the same board imported from a KiCad STEP export.

`checks/` holds 21 numerical self-checks (cavity, line, ports, waveguide port, vias, VNA, STEP import ...).

Open items listed upstream: - wideband dispersive dielectrics (Djordjevic–Sarkar / Debye ADE); - separate merge distance and minimum cell, for narrow CPW gaps; - field monitors and far field.

Why it is useful here

- It is a readable, small FDTD reference next to openEMS (`docs/GDS2OPENEMS.md`) and Hertzon (`docs/HERTZON.md`).

- Its KiCad STEP → FDTD path suits our boards, for example the feed lines of `projects/07_ais140-2w-gnss-tracker`.

Run it

```
cd upstream_snapshot/customcst
python3 -m venv .venv && . .venv/bin/activate
pip install -r requirements.txt   # numpy scipy numba vispy PySide6 cadquery-ocp
python3 main.py                   # GUI (needs a display)
python3 checks/check_fdtd_line.py # one of the numerical self-checks
```

I have not run it here.

Licence and changes

- The upstream is **public domain (The Unlicense)**, see `upstream_snapshot/LICENSE`.
- Changes from upstream: only this README, and `customcst/CLAUDE.md` renamed to `customcst/UPSTREAM_CLAUDE.md`, so that AI coding assistants working in this repository do not load the upstream's agent instructions.
- The sibling folders `atomicfdtd/`, `myinventor/` and `pseudo_cst_3d/` of the upstream repository were not copied.